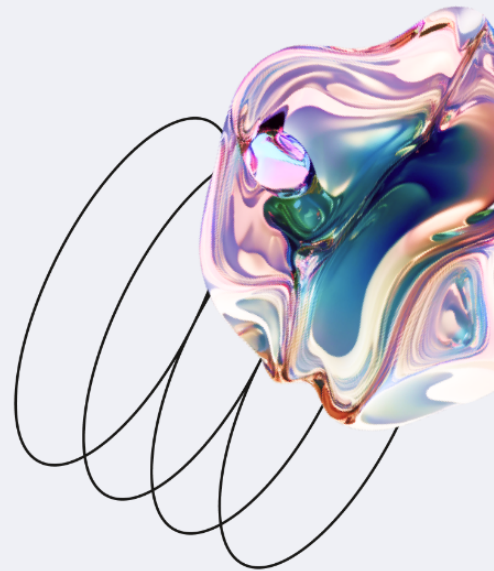




Знакомство со встроенными модулями и фреймворком Express

Основы Node.js



Оглавление

Введение	3
Модули	3
Экспорт данных из модуля	4
Импорт модулей	6
Пользовательские модули	6
Импорт по цепочке	8
Циклические зависимости	9
Глобальные объекты и функции	10
Объект global	11
Объект process	11
Свойства __dirname и __filename	12
Встроенные модули Node.js	12
Модуль fs	12
Метод fs.readFile()	13
Метод .writeFile()	14
Метод fs.appendFile()	15
Синхронные и асинхронные методы	16
Модуль path	17
Модуль os	18
Какие ещё модули существуют?	18
Введение в express	19
Отличие Express от встроенного пакета http	19
Установка и запуск Express	21
Объекты req и res	23
Промежуточные обработчики	23
Глобальные промежуточные обработчики	24
Локальные промежуточные обработчики	26
Переменные в URL	28
Пропуск символов в URL	30
Повторение символов в URL	31
Любые символы в URL	31
Возврат HTML файла	32
Статичные файлы	32
Использование статичных файлов	34
Итоги	35
Домашнее задание	35
Дополнительные материалы	35
Словарь терминов	36

Введение

Вы уже знаете, что запустить файл с JavaScript кодом с помощью Node.js не составляет труда. Для этого достаточно написать в консоли `node` и указать путь к файлу с кодом. Если вы хотите написать небольшую библиотеку, которая, например, содержит в себе функции для работы с математикой, то не составит труда написать все функции в одном файле. Но что если реализуете проект посерьёзнее, например, вы делаете http сервер? В теории можно написать весь код сервера в одном файле и это будет работать, но по мере усложнения проекта кода может стать так много, что будет не очень удобно с ним работать. Вам придётся постоянно листать большой файл в поисках нужной функции или переменной. Плюс ко всему по мере роста проекта неизбежно возникнут ситуации, где бы вам хотелось переиспользовать какую-то часть кода в разных местах. Для того чтобы решить этот вопрос, в Node.js существуют модули, которые позволяют разделить проект на несколько файлов и переиспользовать функции и переменные. Давайте попробуем разобраться, как это реализовано в Node.js.

Модули

Для того чтобы разделить проект Node.js на несколько файлов, необходимо понимать, каким образом обеспечивается экспорт и импорт данных в модулях. Вы уже частично знакомы с функцией `require()` и с глобальным объектом `module.exports`. Давайте более подробно разберём их работу. Прежде чем приступим, дадим определение модулю:



Модуль – это файл, код которого можно использовать в других модулях.

Существует три вида модулей:

- Встроенные модули – эти модули являются частью Node.js. Вы уже знакомы с одним из таких модулей – это модуль `http`.
- Модули NPM – эти модули можно установить с помощью NPM. С такими модулями вы тоже познакомились во второй лекции.
- Пользовательские модули – это модули, которые вы создаёте сами. С этим видом модулей мы познакомимся более подробно в этой лекции.

Экспорт данных из модуля

Любой модуль может экспортировать какие-либо данные, чтобы их можно было переиспользовать в другом модуле. Если представить модуль, как обычную функцию, то экспорт из модуля – это аналог оператора `return`. То есть модуль возвращает какие-то значения во время импорта этого модуля. В Node.js экспорт из модуля реализуется через уже знакомый вам глобальный объект `module.exports`.

Ниже представлен пример экспорта различных данных:

```
1 function func() {  
2   console.log('Hello!');  
3 }  
4  
5 const string = 'some string';  
6  
7 const number = 10;  
8  
9 const array = [1, 2, 3];  
10  
11 const object = { key: 'value' };  
12  
13 module.exports = { func, string, number, array, object };
```

Из модуля можно экспортировать любые типы данных: функции, строки, числа, массивы и объекты. По умолчанию `module.exports` – это пустой объект. В примере выше мы переопределили этот пустой объект другим объектом с различными свойствами. Теперь этот модуль можно импортировать в другом модуле и использовать эти свойства по своему усмотрению.

Необходимо помнить одно правило – экспорт должен происходить незамедлительно! Давайте разберём это правило на примере:

```
1 setTimeout(() => {  
2   const someVar = 10;  
3   module.exports = { someVar };  
4 }, 0);
```

`module.exports` определён в колбеке `setTimeout`. Так как `setTimeout` асинхронная функция, то колбек выполнится в будущем. Это означает, что как только мы импортируем модуль с таким кодом, мы не получим в `module.exports` нужные

данные, они определяются чуть позже. Именно по этому нельзя использовать `module.exports` в асинхронных функциях.

Вот ещё пример, когда экспорт не будет работать:

```
1 function exportInFunction() {  
2     const someVar = 10;  
3     module.exports = { someVar };  
4 }
```

Тут в целом понятно, почему не получится экспортировать данные, так как функция не вызывается. Если функцию вызвать, то данные будут экспортированы:

```
1 function exportInFunction() {  
2     const someVar = 10;  
3     module.exports = { someVar };  
4 }  
5  
6 exportInFunction();
```

Этот способ рабочий, но так использовать `module.exports` не рекомендуется, так как это будет вас путать.

💡 Хорошим способом экспортировать данные из модуля будет определение `module.exports` в конце скрипта.

Также модуль может не экспортировать данные:

```
1 console.log('Module without exports!');
```

Если вы импортируете такой модуль, то вы получите в консоли сообщение из `console.log`. То есть модуль просто исполнится и ничего не вернёт в качестве экспорта.

Такие модули редко бывают полезными, но важно понимать такое свойство модулей.

Импорт модулей

С импортом вы уже знакомы. Импортировать модули можно с помощью функции `require()`.

💡 Функция **`require()`** – это глобальная функция Node.js, которая используется для загрузки и кэширования модулей. Она принимает один аргумент – имя или путь к файлу модуля, который нужно загрузить, и возвращает экспортированные свойства из этого модуля.

Ниже приведены примеры импорта различных видов модулей:

```
1 // Импорт глобального модуля
2 const http = require('http');
3
4 // Импорт NPM модуля
5 const uuid = require('uuid');
6
7 // Импорт пользовательского модуля
8 const myModule = require('./myModule');
```

Функция `require()` может принять в качестве аргумента имя модуля, если это глобальный модуль или модуль NPM, либо путь к модулю, если это пользовательский модуль.

Функция **`require()`** работает следующим образом:

1. Функция `require()` сначала ищет встроенные модули по переданному имени модуля.
2. Если глобального модуля с указанным именем нет, то `require()` ищет установленные модули с помощью NPM.
3. И наконец, если нет ни глобального, ни NPM модулей, или указан путь к модулю, то ищет среди пользовательских модулей.

Это упрощенный алгоритм работы функции. Если хотите ознакомиться более подробно с алгоритмом, можете обратиться к [официальной документации](#).

Пользовательские модули

Как мы уже выяснили, можно создавать собственные модули. Фактически это отдельные файлы в проекте. Чтобы импортировать пользовательский модуль, достаточно в аргумент функции `require()` указать путь к файлу:

```
1 // Импорт модуля из текущей директории
2 const firstModule = require('./firstModule');
3 // Импорт модуля из внутренней директории
4 const secondModule = require('./test/secondModule');
5 // Импорт модуля из внешней директории
6 const thirdModule = require('../dist/thirdModule');
7 // Импорт модуля с расширением файла
8 const moduleWithExtention = require('./moduleWithExtention.js');
```

Обратите внимание, что путь к модулю указывается относительно модуля, в котором мы импортируем. Также можно не указывать расширение файла модуля, `require()` автоматически подставит расширение, если его нет.

Ещё `require()` позволяет импортировать директорию, вместо файла. Для этого необходимо указать путь к директории и в директории обязательно должен находиться файл `index.js`:

```
1 // ./myLib/index.js
2 module.exports = { hello: 'hello' };
3
4 // main.js
5 const myLib = require('./myLib')
```

Выше приведён пример того, как можно импортировать директорию. В директории `myLib` находится файл `index.js`, который экспортирует объект со свойством `hello`.

Импортируется эта директория в файле `main.js` и в `require()` указывается путь до директории импортируемого модуля.

Такой импорт может пригодиться, когда вы реализуете, к примеру, библиотеку, которая разбита на несколько файлов. Для того чтобы не думать при импорте, какой конкретно файл нужно указывать, вы просто можете создать директорию и определить в ней файл `index.js`, который будет использоваться для импорта по умолчанию и при импорте указывать просто путь к директории.

Импорт по цепочке

Важно понимать, что импортируемые модули могут также импортировать в себя другие модули. Ниже приведён пример цепочки импортов:

Первый модуль в этой цепочке – это модуль `baz`, который ничего не импортирует, но экспортирует данные.

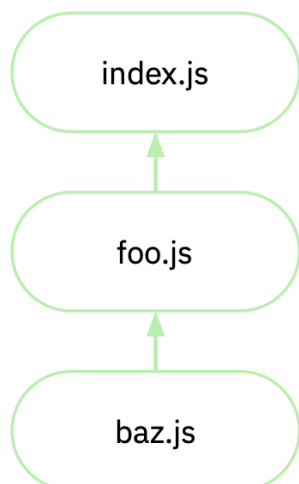
```
1 // baz.js
2 module.exports = 'Hello from baz!';
```

Второй модуль – это модуль `foo`, который импортирует модуль `baz` и экспортирует собственные данные.

```
1 // foo.js
2 const baz = require('./baz');
3
4 console.log(baz);
5
6 module.exports = 'Hello form foo!';
```

И третий модуль – это модуль `index`, который импортирует модуль `foo`

```
1 // index.js
2 const foo = require('./foo');
3
4 console.log(foo);
```



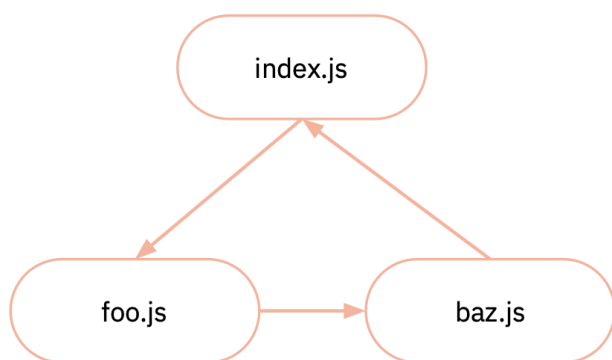
Так выглядит вся эта цепочка:

`index.js` импортирует `foo.js`, который, в свою очередь, импортирует `baz.js`.

Цепочки импортов могут быть довольно большими. Но есть один случай, при котором цепочка импортов может не работать. Об этом случае отдельно поговорим ниже.

Циклические зависимости

Иногда могут возникнуть ситуации, когда цепочка зависимостей начинается и заканчивается на одном и том же файле. Это называется циклические зависимости. Ниже представлен пример таких зависимостей:



В данном примере index.js импортирует foo.js. foo.js, в свою очередь, импортирует baz.js, а baz.js импортирует index.js.

То есть цепочка импортов начинается и заканчивается на одном и том же файле – index.js.

Давайте рассмотрим на примере кода:

```
1 // foo.js
2 const baz = require('./baz');
3
4 console.log(baz);
5
6 module.exports = { hello: 'hello from foo' };
```

Допустим, есть модуль foo.js, который импортирует модуль baz.js. При этом foo.js экспортирует объект со свойством hello.

```
1 // baz.js
2 const foo = require('./foo');
3
4 console.log(foo);
5
6 module.exports = { hello: 'hello from baz' };
```

И есть модуль baz.js, который импортирует модуль foo.js и экспортирует объект со свойством hello.

Если запустить модуль foo.js, то мы получим примерно такой вывод в консоли:

```
1 {}
2 { hello: 'hello from foo' }
3 (node:14380) Warning: Accessing non-existent property
  'Symbol(nodejs.util.inspect.custom)' of module exports inside
  circular dependency
4 (Use `node --trace-warnings ...` to show where the warning was
  created)
5 (node:14380) Warning: Accessing non-existent property
  'constructor' of module exports inside circular dependency
6 (node:14380) Warning: Accessing non-existent property
  'Symbol(Symbol.toStringTag)' of module exports inside circular
  dependency
7 (node:14380) Warning: Accessing non-existent property
  'Symbol(Symbol.iterator)' of module exports inside circular
  dependency
```

На первой строчке выведется пустой объект, на второй объект, который экспортируется из модуля `foo.js`, а дальше мы увидим текст предупреждения, который говорит о том, что в коде есть циклические зависимости.

Почему же мы не получили объект, у которого в свойстве `hello` текст “hello from baz”, а вместо этого получили пустой объект?

Когда запускается модуль `foo.js`, он тут же импортирует модуль `baz.js`. `baz.js`, в свою очередь, импортирует `foo.js` и выводит в консоль экспорт модуля `foo.js`. Так как модуль `foo.js` ещё не инициализирован, то `require()` в `baz.js` возвращает пустой объект экспорта (значение по умолчанию). Именно поэтому в консоли мы увидели пустой объект.

Таких зависимостей стоит избегать, так как они ломают работу кода.

Глобальные объекты и функции

Для удобства некоторая функциональность Node.js реализована не в отдельном модуле, а представлена глобально. Что это значит?

Глобальные объекты и функции Node.js – это объекты и функции, которые доступны во всех модулях без необходимости их подключения с помощью `require()`. Например, **`console`**, **`process`** и **`__dirname`** и т. д. Они являются частью среды исполнения Node.js и определены как глобальные объекты и функции для удобства.

Для использования глобальных объектов и функций Node.js достаточно просто обратиться к ним по имени. Например:

```
1 // Выводит "Hello, world!" в стандартный поток вывода
2 console.log("Hello, world!");
3
4 // Возвращает текущую версию Node.js
5 console.log(process.version);
6
7 // Загружает модуль fs для работы с файловой системой
8 const fs = require("fs");
```

Объект global

В Node.js существует глобальный объект `global`. В этом объекте хранятся различные объекты и функции, такие как `console`, `Math`, `setTimeout` и т. д.

Тут вы можете прийти в недоумение, так как `console` можно вызывать из любого места и не нужно обращаться к `global`. Да, все так! Это возможно потому, что объект `global` записывается в глобальный контекст при старте скрипта и именно поэтому все его свойства и функции доступны без обращения к самому объекту `global`. Но ничего не мешает вам обратиться через `global`, например, так:

```
1 global.Math.floor(0.5);
2 // Тоже самое, что и
3 Math.floor(0.5);
4
5 global.setTimeout(() => console.log('Hello, World!!!'), 1000);
6 // Тоже самое, что и
7 setTimeout(() => console.log('Hello, World!!!'), 1000);
```

Объект process

Ещё в Node.js существует объект `process`. Он содержит в себе набор функций и свойств для работы с операционной системой и запущенным процессом Node.js.

```
1 console.log(process.version); // Версия Node.js
2 console.log(process.arch); // Архитектура процессора
3 console.log(process.pid); // Директория откуда запущен скрипт
4 console.log(process.cwd()); // Директория откуда запущен скрипт
```

Свойства `__dirname` и `__filename`

Для того чтобы из скрипта определить путь в файловой системе к этому же скрипту, можно воспользоваться двумя свойствами: `__dirname` и `__filename`.

```
1 // Предположим, что скрипт находится тут:  
  /Users/Student/Desktop/index.js  
2  
3 // Выведет в консоль путь к файлу  
4 // /Users/Student/Desktop/index.js  
5 console.log(__filename);  
6  
7 // Выведет в консоль путь к директории хранения файла  
8 // /Users/Student/Desktop  
9 console.log(__dirname);
```

Обратите внимание на разницу между `process.cwd()` и `__dirname`. С первого взгляда кажется, что они делают одно и то же, но на самом деле между ними есть существенное различие: `process.cwd()` возвращает путь к директории, откуда был запущен скрипт, а `__dirname` возвращает директорию, где скрипт хранится.

Это не все глобальные получения полного списка глобальных объектов и функций Node.js можно обратиться к [официальной документации](#).

Встроенные модули Node.js

Вы уже знаете, что встроенные модули устанавливаются вместе с Node.js и их не нужно устанавливать отдельно. Встроенные модули предоставляют различные возможности для работы с файлами, сетью, операционной системой, потоками, событиями, криптографией и другими аспектами программирования на Node.js. Вы уже знакомы с модулем `http`, и в этой лекции мы рассмотрим ещё три полезных модуля – это модули `fs`, `path` и `os`.

Модуль `fs`

Модуль **fs** (file system) предоставляет функции для работы с файловой системой. С его помощью можно читать, записывать, удалять и перемещать файлы и директории. После подключения модуля мы можем использовать его методы для работы с файлами.

Метод fs.readFile()

Для чтения файлов существует метод `fs.readFile()`. Он позволяет получить содержимое любого файла:

```
1 const fs = require('fs');
2
3 // читаем файл hello.txt
4 fs.readFile('./hello.txt', 'utf8', (err, data) => {
5   if (err) {
6     // если произошла ошибка, выводим ее
7     console.error(err);
8   } else {
9     // если нет ошибки, выводим содержимое файла
10    console.log(data);
11  }
12 });
13
```

Метод принимает три аргумента:

- путь к файлу, который хотим прочитать;
- кодировку файла (опционально);
- функцию обратного вызова (callback), которая будет вызвана после чтения файла.

Функция обратного вызова в третьем аргументе имеет два аргумента:

- объект ошибки (если произошла ошибка при чтении файла);
- данные файла (если чтение файла успешно).

Метод **`fs.readFile()`** является асинхронным, то есть он не блокирует выполнение остального кода до тех пор, пока не прочитает файл. Вместо этого он передаёт результат чтения файла в функцию обратного вызова, которая будет вызвана позже.

Теперь, когда вы познакомились с методом, попробуйте ответить на вопрос. Что выведет в консоль скрипт ниже?

```

1 const fs = require('fs');
2
3 fs.readFile(__filename, 'utf-8', (err, data) => {
4     if (err) {
5         console.error(err);
6     } else {
7         console.log(data);
8     }
9 });

```

Ответ: в консоли вы увидите содержимое файла скрипта, так как глобальное свойство `__filename` хранит в себе путь к запущенному скрипту и, следовательно, метод `.readFile()` прочитает этот же скрипт и выведет текст скрипта в консоль.

Метод `.writeFile()`

Чтобы записать что-то в файл, можно воспользоваться методом `.writeFile()`. Этот метод полностью перезаписывает указанный файл:

```

1 const fs = require('fs');
2
3 fs.writeFile('/path/to/file', 'Some text data', (err) => {
4     if (err) {
5         console.error(err);
6     }
7     console.log('The file was saved!');
8 });

```

Метод принимает три аргумента:

- путь к файлу, в который нужно записать данные;
- строка с данными, которые хотим записать;
- функция обратного вызова, которая запустится, когда файл будет записан.

Функция обратного вызова в третьем аргументе принимает один аргумент:

- объект ошибки, который будет инициализирован в случае ошибки записи.

Теперь попробуйте ответить на такой вопрос: какой будет результат выполнения скрипта ниже?

```
1 const fs = require('fs');
2
3 fs.writeFile(__filename, 'console.log("Hello!")', (err) => {
4   if (err) {
5     console.error(err);
6   }
7   console.log('The file was saved!');
8 });
```



Ответ: будет перезаписан запущенный файл скрипта. Здесь так же, как и в примере с `fs.readFile()`, `__filename` указывает на текущий скрипт и, так как `fs.writeFile()` перезаписывает файл полностью, в итоге вся информация в скрипте удалится и появится `console.log("Hello!")`.

Но что, если хочется не перезаписывать файл, а добавлять в файл данные? Для этого существует метод `fs.appendFile()`.

Метод `fs.appendFile()`

Как и говорилось выше, метод `fs.appendFile()` добавляет данные в файл и не перезаписывает его полностью. Этот метод работает очень похожим образом, как и `fs.writeFile()`. Давайте рассмотрим пример:

```
1 const fs = require('fs');
2
3 fs.appendFile('/path/to/file', 'console.log("Hello!")', (err) => {
4     if (err) {
5         console.error(err);
6     }
7
8     console.log('The file was saved!');
9 });
```

Метод `fs.appendFile()` принимает 3 аргумента:

- путь к файлу, в который хотим добавить данные;
- данные, которые нужно добавить в файл;
- колбек, который отработает, после того, как будут записаны данные в файл.

Функция обратного вызова в третьем аргументе принимает один аргумент:

- объект ошибки, который будет инициализирован в случае ошибки записи.

Синхронные и асинхронные методы

Модуль `fs` помимо асинхронных методов имеет также синхронные методы. Давайте разберёмся в чём различие.

Асинхронные методы имеют преимущество в том, что они не замедляют работу приложения и позволяют обрабатывать множество файлов одновременно. Однако, это также означает, что порядок выполнения кода может быть неочевидным.

Синхронные же методы блокируют выполнение скрипта до тех пор, пока работа с файлом не закончится и при этом не требуют определения колбека в третьем аргументе. Также методы называются по-другому и имеют постфикс `Sync`. Например, синхронный аналог метода `fs.readFile()` является `fs.readFileSync()`, а аналогом метода `fs.appendFile()` является `fs.appendFileSync()`.

Давайте рассмотрим пример:


```
1 const fs = require('fs');
2
3 try {
4     const result = fs.readFileSync(__filename, 'utf-8');
5     console.log(result);
6 } catch(err) {
7     console.error(err);
8 }
9
10 try {
11     fs.appendFileSync(__filename, 'console.log("Hello!")');
12     console.log('The file was saved!');
13 } catch(err) {
14     console.error(err);
15 }
```

Как видите, результат чтения файла возвращается функцией `fs.readFileSync()`, а любые ошибки, которые могут возникнуть во время работы с файлом, необходимо обрабатывать с помощью `try {} catch {}`.

В целом синхронные методы рекомендуется использовать только в тех случаях, когда вы уверены, что они не повлияют на работу вашего приложения. Например, если вы делаете http сервер, который постоянно слушает входящие запросы, не стоит использовать синхронные функции, так как они остановят выполнение вашего кода и ваш сервер не сможет обработать входящие запросы, пока работа с файлом не будет закончена.

Синхронные методы часто используют при старте приложения, например, чтобы один раз прочитать какие-нибудь конфигурационные файлы. Или когда реализуют простой скрипт, который не требует высокой скорости работы. Это допустимые сценарии использования. Во всех остальных случаях следует воздержаться от использования синхронных методов.

Модуль path

Часто при работе с файлами возникает необходимость составлять пути к файлам из нескольких частей. Это можно делать с помощью конкатенации строк, но более надёжным вариантом является использование модуля `path`.

Модуль `path` предоставляет ряд методов для работы с путями в файловой системе. Ниже приведён пример работы некоторых методов:

- Метод **path.join()** позволяет объединить две части путей в один путь. Метод вернёт строку с результатом объединения.

```
1 const path = require('path');
2
3 console.log(path.join('/User/Student', 'Desktop/index.js'));
4 // Вернёт строку "/User/Student/Desktop/index.js"
```

- Метод **path.parse()** возвращает объект пути, в котором хранятся составляющие пути.

```
1 const path = require('path');
2
3 console.log(path.parse('/User/Student/Desktop/index.js'));
```

Вызов метода из примера выше вернёт следующий объект:

```
1 {
2   root: '/',
3   dir: '/User/Student/Desktop',
4   base: 'index.js',
5   ext: '.js',
6   name: 'index'
7 }
```

- Метод **path.dirname()** возвращает путь к директории, исключив название файла из пути.

```
1 const path = require('path');
2
3 console.log(path.dirname('/User/Student/Desktop/index.js'));
4 // Вернется строка /User/Student/Desktop
```

- Метод **path.extname()** возвращает расширение файла, который указан в передаваемом пути.

```
1 const path = require('path');
2
3 console.log(path.extname('/User/Student/Desktop/index.js'));
4 // Вернется строка со значением ".js"
```

Модуль os

Модуль OS позволяет получить информацию об операционной системе

```
1 const os = require('os');
2
3 // Вернет массив с информацией о ядрах процессора
4 console.log(os.cpus());
5 // Вернет информацию об архитектуре процессора
6 console.log(os.arch());
7 // Вернет информацию о совокупной оперативной памяти
8 console.log(os.freemem());
9 // Вернет информацию об общем количестве оперативной памяти в МВ
10 console.log(os.totalmem() / 1024 / 1024 / 1024);
```

Какие ещё модули существуют?

url – позволяет работать с URL адресами и их компонентами;

crypto – позволяет работать с криптографией. Содержит в себе различные криптографические функции;

stream – позволяет управлять большими потоками данных.

А также многие другие модули.

В Node.js довольно много модулей, и не все модули вам могут пригодиться в ваших проектах. Но я рекомендую вам ознакомиться со всеми модулями, которые есть в Node.js. Для этого можно обратиться к [официальной документации](#).

Введение в express

Теперь давайте попробуем использовать сторонний модуль, который поможет нам реализовать http сервер.

Для реализации веб-сервера, конечно же, можно использовать встроенный модуль http, но он довольно низкоуровневый и в случае, если вы сделаете действительно большой и сложный веб-сервер, код будет читать довольно сложно. В связи с этим существует множество сторонних библиотек и фреймворков, которые помогают упростить код и предоставить более высокий уровень абстракции для реализации http сервера. Одним из таких фреймворков – это express.

Express – это фреймворк для Node.js, который предоставляет набор функций и инструментов для создания веб-приложений и API.

Express позволяет легко и быстро создавать веб-сервер, обрабатывать HTTP-запросы, маршрутизировать URL, работать с промежуточными обработчиками (middleware), использовать шаблонизаторы для генерации HTML-страниц, настраивать параметры и переменные окружения, обрабатывать ошибки и многое другое. Express является минималистичным и гибким фреймворком, который не навязывает определённой структуры или способа организации кода, а позволяет разработчику самостоятельно выбирать нужные модули и библиотеки.

Отличие Express от встроенного пакета http

Express построен на основе встроенного пакета http, который предоставляет низкоуровневые функции для работы с HTTP-протоколом. Express расширяет и упрощает возможности пакета http, добавляя высокоуровневые абстракции и удобные методы для создания веб-приложений. Например, с помощью Express можно легко создать http сервер таким образом:

```
1 // Подключаем модуль express
2 const express = require('express');
3
4 // Создаем объект приложения
5 const app = express();
6
7 // Создаем обработчик GET-запроса по корневому пути
8 app.get('/', function(req, res) {
9   res.send('Hello World'); // отправляем ответ клиенту
10 });
11
12 // Запускаем сервер на порту 3000
13 app.listen(3000);
14
```

А с помощью пакета http это потребовало бы больше кода:

```

1 // Подключаем модуль http
2 const http = require('http');
3
4 // Создаем объект сервера
5 const server = http.createServer(function(req, res) {
6   // Проверяем метод и путь запроса
7   if (req.method === 'GET' && req.url === '/') {
8     // Устанавливаем заголовки ответа
9     res.writeHead(200, {'Content-Type': 'text/plain'});
10    // Отправляем тело ответа
11    res.end('Hello World');
12  }
13 });
14
15 // Запускаем сервер на порту 3000
16 server.listen(3000);
17

```

Express позволяет:

- Использовать методы **app.get()**, **app.post()** и т. д. для определения обработчиков запросов по разным HTTP-методам и URL-путям.
- Использовать метод **res.send()** для отправки ответа с автоматическим определением типа контента и кодировки.
- Использовать метод **app.listen()** для запуска сервера на указанном порту.

Кроме того, Express предоставляет множество других возможностей, таких как:

- Поддержка параметров в URL-путях. Например, если вы хотите определить часть URL, как изменяемую часть.
- Поддержка статических файлов и шаблонизаторов.
- Поддержка промежуточных обработчиков для выполнения различных задач перед или после обработки запроса.
- Поддержка маршрутизаторов для организации обработчиков по группам или модулям.
- Поддержка обработки ошибок и специальных ответов (например, 404).

Таким образом, Express значительно упрощает и улучшает работу с HTTP-серверами на Node.js по сравнению со встроенным пакетом http.

Установка и запуск Express

Для того чтобы использовать Express в своём проекте, необходимо установить его как зависимость с помощью NPM. Для этого нужно выполнить следующую команду в терминале:

```
1 npm install express
```

Эта команда установит Express в папку **node_modules** и добавит его в файл **package.json** в раздел **dependencies**.

Для того чтобы использовать Express в своём коде, необходимо импортировать его с помощью функции **require()** и создать экземпляр приложения с помощью функции **express()**:

```
1 const express = require('express');  
2 const app = express();
```

Экземпляр приложения **app** представляет собой объект, который содержит методы для настройки и запуска сервера, а также для определения обработчиков запросов.

Далее необходимо запустить сервер с помощью метода `app.listen()`. Этот метод работает так же, как и метод `.listen()` у встроенного модуля `http`. Для запуска, необходимо передать в аргумент метода `.listen()` номер порта, на котором будет слушать сервер.

```
1 const express = require('express');  
2 const app = express();  
3  
4 app.listen(3000);
```

Теперь можно определять HTTP обработчики. Это делается вызовом методов `.get()`, `.post()`, `.put()`, `.delete()` и т. д. на объекте `app`.

```
1 const express = require('express');
2 const app = express();
3
4 app.get('/', (req, res) => {
5   res.send("<h1>Hello, World!!!</h1>");
6 });
7
8 app.listen(3000);
```

Определять обработчики можно как до вызова метода `.listen()`, так и после. Работать будет одинаково в большинстве случаев. Но лучше определять обработчики до старта сервера, чтобы избежать случаев, когда сервер запущен, но обработчики ещё не инициализировались.

Express позволяет определить несколько обработчиков. Например, мы можем определить обработчик для роута `/about`:

```
1 const express = require('express');
2 const app = express();
3
4 app.get('/', (req, res) => {
5   res.send("<h1>Hello, World!!!</h1>");
6 });
7
8 app.get('/about', () => {
9   res.send("<h1>About page</h1>");
10 });
11
12 app.listen(3000);
```

Вы могли обратить внимание, что в примерах реализуются обработчики только для HTTP метода GET. В этой лекции мы будем разбирать примеры именно с этим методом, так как такие обработчики легко проверить в браузере. В следующей лекции вы познакомитесь с обработчиками других методов.

Объекты req и res

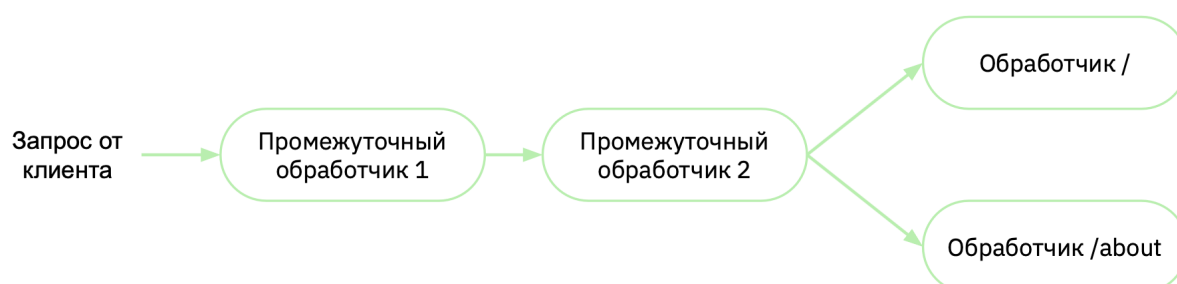
Давайте теперь подробнее разберём колбек в третьем аргументе. Этот колбек выполняется, когда приходит запрос и в аргументах этого колбека мы можем получить два объекта.

Первый объект – это объект req (request). Этот объект хранит в себе информацию о запросе, такую как: url запроса, метод запроса, информация о клиенте и т. д.

Второй объект – это объект res (response). Этот объект хранит в себе свойства и методы, которые позволяют работать с ответом. Например, метод `.send()` позволяет завершить запрос и указать данные, которые необходимо вернуть клиенту, а метод `.status()` позволяет указать http статус ответа.

Промежуточные обработчики

В Express существует концепция промежуточных обработчиков или middleware. Такие обработчики нужны, для того чтобы задавать дополнительную логику для основных обработчиков.



На схеме выше вы можете увидеть, как могут применяться промежуточные обработчики. На ней видно, что прежде чем запустится один из основных обработчиков роутов, будут запущены два промежуточных обработчика.

Существует два вида промежуточных обработчиков: глобальные и локальные.

- Глобальные промежуточные обработчики не привязаны к конкретному роуту и обрабатывают для группы роутов или абсолютно для всех роутов.
- Локальные промежуточные обработчики определяются для конкретного роута.

Глобальные промежуточные обработчики

Давайте на примере разберём работу глобальных промежуточных обработчиков:

```
1 const express = require('express');
2
3 const app = express();
4
5 // Логируем факт запроса
6 app.use((req, res, next) => {
7   console.log('Поступил запрос', req.method, req.url);
8   next();
9 });
10
11 app.get('/', (req, res) => {
12   res.send('<h1>Добро пожаловать на мой сайт!</h1>');
13 });
14
15 app.get('/about', (req, res) => {
16   res.send('<h1>Страница обо мне</h1>');
17 });
18
19 app.listen(3000, () => {
20   console.log(`Сервер запущен на порту ${port}`);
21 });
```

Допустим, мы хотим логировать любой входящий запрос. Чтобы не писать логику логирования в каждом роуте, можно добавить глобальный промежуточный обработчик до определения всех роутов. Реализуется это с помощью метода `app.use()`. На шестой строке вы можете увидеть, что метод принимает один единственный аргумент в виде колбека, который очень похож на колбек в обычном обработчике. В первом аргументе можно получить объект запроса, во втором – объект ответа, а третий аргумент хранит в себе функцию `next()`.

Функция `next()` необходима для того чтобы продолжить вызов следующих обработчиков. Если функцию `next()` не вызвать, то обработчики роутов не запустятся и запрос зависнет. То есть в промежуточном обработчике мы должны сообщить `express`у`, когда стоит запускать следующие обработчики.

🔥 Важно понимать, что порядок определения обработчиков очень важен! Например, если в примере выше мы определим промежуточный обработчик после основных обработчиков роутов, то промежуточный обработчик никогда не выполнится.

```
1 const express = require('express');
2
3 const app = express();
4
5 app.get('/', (req, res) => {
6   res.send('<h1>Добро пожаловать на мой сайт!</h1>');
7 });
8
9 app.get('/about', (req, res) => {
10   res.send('<h1>Страница обо мне</h1>');
11 });
12
13 // Логируем факт запроса
14 app.use((req, res, next) => {
15   console.log('Поступил запрос', req.method, req.url);
16   next();
17 });
18
19 app.listen(3000, () => {
20   console.log(`Сервер запущен на порту ${port}`);
21 });
```

В коде выше не будет работать, как ожидается, так как обработка запроса начнётся в первую очередь в обработчике конкретного роута и там же запрос завершится отправкой HTML клиенту и промежуточный обработчик никогда не выполнится.

Но существуют ситуации, когда промежуточный обработчик, определённый после основных обработчиков роутов, будет работать. Это может произойти в том случае, если не был найден нужный обработчик роута. Например:

```
1 const express = require('express');
2 const app = express();
3
4
5 app.get('/', (req, res) => {
6   res.send("<h1>Hello, World!!!</h1>");
7 });
8
9 app.use((req, res) => {
10   res.status(404);
11   res.send('<h1>Страница не найдена!</h1>');
12 });
13
14 app.listen(3000);
```

В коде выше на строке пять определён один единственный обработчик для корневого роута. После определения этого обработчика, на строке девять определён промежуточный обработчик, в котором устанавливается статус ответа `res.status(404)`; и отправляется HTML клиенту. Если запустить этот код и попробовать обратиться к неопределённому роуту, например, к `/about`, то запустится промежуточный обработчик на девятой строке. Это происходит потому, что `express` ищет обработчик для переданного `url`, и если такого обработчика нет, то запускается следующий глобальный обработчик по порядку. В нашем случае этот обработчик отправляет информацию клиенту о том, что страница не найдена.

Вот так просто в `express` реализовать обработку несуществующих роутов, для этого достаточно определить глобальный обработчик после всех основных.

Локальные промежуточные обработчики

Теперь давайте рассмотрим локальные промежуточные обработчики, которые влияют на конкретные роуты:

```

1 const express = require('express');
2 const app = express();
3
4 function logRequest(req, res, next) {
5   console.log('Поступил запрос', req.method, req.url);
6   next();
7 }
8
9 app.get('/', logRequest, (req, res) => {
10   res.send("<h1>Hello, World!!!</h1>");
11 });
12
13 app.listen(3000);

```

В примере выше на четвёртой строке определена функция промежуточного обработчика, которая логирует запрос. Далее обратите внимание на девятую строку. Там, определённый выше промежуточный обработчик, передан во второй аргумент, а в третьем уже определён основной обработчик. На самом деле express, при вызове методов определения обработчиков, принимает любое количество аргументов. То есть начиная со второго аргумента, express вызывает все указанные далее в аргументах обработчики по порядку.

```

1 const express = require('express');
2 const app = express();
3
4 function logRequest(req, res, next) {
5   console.log('Поступил запрос', req.method, req.url);
6   next();
7 }
8
9 app.get('/', logRequest, logRequest, logRequest, (req, res) => {
10   res.send("<h1>Hello, World!!!</h1>");
11 });
12
13 app.listen(3000);

```

В примере выше переданы друг за другом три промежуточных обработчика, и только потом определён основной обработчик роута. Запустив этот код, при запросе на ваш роут, вы получите три сообщения в консоли.

Также можно определить промежуточный обработчик прямо в вызове метода, как и основной обработчик:

```

1 const express = require('express');
2 const app = express();
3
4 app.get(
5   '/',
6   (req, res, next) => {
7     console.log('Поступил запрос', req.method, req.url);
8     next();
9   },
10  (req, res) => {
11    res.send("<h1>Hello, World!!!</h1>");
12  }
13 );
14
15 app.listen(3000);

```

Локальные промежуточные обработчики позволяют определить уникальное поведение для определённых роутов. Например, аутентификацию запроса можно реализовать с помощью локального промежуточного обработчика, один раз определив функцию такого обработчика и добавляя её только в те роуты, где пользователи должны быть аутентифицированы.

Переменные в URL

Иногда вам может понадобиться иметь динамическую часть URL. Например, когда вы реализуете сайт со статьями, где номер каждой статьи определён в URL, к пример так: `/article/83774`. В таком URL динамической частью является идентификатор статьи. Заранее, скорее всего, вам не будет известно сколько будет статей и какие у них будут идентификаторы, и делать под каждую статью обработчик будет довольно затратно, поэтому в express существует механизм URL переменных. Давайте рассмотрим пример кода:

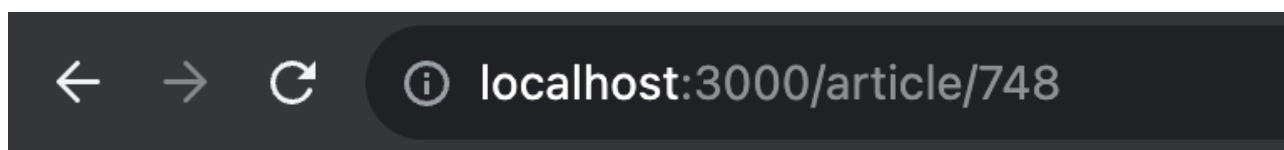
```

1 const express = require('express');
2 const app = express();
3
4 app.get('/article/:id', (req, res) => {
5   res.send(`<h1>Статья №${req.params.id}</h1>`);
6 });
7
8 app.listen(3000);

```

Обратите внимание на четвёртую строку и первый аргумент. В него передаётся строка “/article/:id”, где :id express расценивает, как динамическую часть URL. Динамическую часть URL вы можете именовать как угодно, например, /article/:articleID. Таким образом в браузере можно сделать запрос на сервер с таким URL /article/748 и express запустит, определённый на четвёртой строке, обработчик.

Далее посмотрите на пятую строку. Там мы возвращаем клиенту заголовок, в который передаем номер статьи из URL. Данные, указанные в динамической части URL, хранятся в объекте req в свойстве params. Получить данные, которые были переданы из браузера, можно по названию, который вы определили в URL – это id. Если вы сделаете запрос с URL /article/748, то вот что вы увидите в браузере:

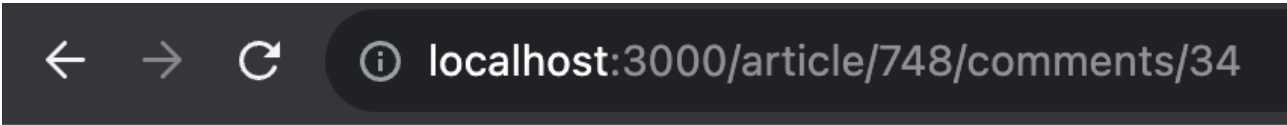


Статья №748

Вы можете определить несколько переменных в URL. Предположим, что под статьями можно оставлять комментарии и у каждого комментария свой идентификатор. В таком случае, чтобы получить текст какого-то конкретного комментария, необходимо определить следующий роут: /article/:articleID/comments/:commentID. Давайте разберём на примере:

```
1 const express = require('express');
2 const app = express();
3
4 app.get(
5   '/article/:articleID/comments/:commentID',
6   (req, res) => {
7     const html = `
8       <h1>Статья №${req.params.articleID}</h1>
9       <p>Комментарий №${req.params.commentID}</p>
10     `
11     res.send(html);
12   },
13 );
14
15 app.listen(3000);
```

Номер статьи можно получить в свойстве `req.params.articleID`, а номер комментария в свойстве `req.params.commentID`. Запустив такой сервер и сделав запрос, в браузере можно получить примерно следующее:



← → ↻ ⓘ localhost:3000/article/748/comments/34

Статья №748

Комментарий №34

Пропуск символов в URL

Ещё иногда вам может понадобиться сделать ваши URL более “дружелюбными” для пользователей. Например, когда вы знаете, что пользователь может ошибиться в URL, к примеру, в слове `comment` написать одну букву `m`. Для таких случаев `express` позволяет пропускать символы. Давайте посмотрим пример:

```
1 const express = require('express');
2
3 const app = express();
4
5 app.get('/com?ment', (req, res) => {
6   res.send('/com?ment');
7 });
8
9 app.listen(3000, () => {
10   console.log(`Сервер запущен на порту ${port}`);
11 });
```

На пятой строке в первом аргументе в слове `comment` указан специальный символ “?”, который позволяет сделать запрос как на URL `/comment`, так и на `/coment`. Знак вопроса говорит `express` о том, что последующий за этим знаком символ может быть пропущен.

Повторение символов в URL

Также можно указать, какие символы могут быть повторены несколько раз. Для этого необходимо использовать знак “+”. Например:

```
1 const express = require('express');
2
3 const app = express();
4
5 app.get('/add+ress', (req, res) => {
6   res.send('/add+ress');
7 });
8
9 app.listen(3000, () => {
10   console.log(`Сервер запущен на порту ${port}`);
11 });
```

На пятой строке в первом аргументе в слове address добавлен знак + после второй буквы d, и это означает, что предыдущий символ в URL можно указывать сколько угодно раз. Например, так address или так addddddddress.

Любые символы в URL

Ещё есть возможность в URL, что могут быть повторены любые символы. Для этого необходимо использовать знак “*”. Например:

```
1 const express = require('express');
2
3 const app = express();
4
5 app.get('/add*ress', (req, res) => {
6   res.send('/add*ress');
7 });
8
9 app.listen(3000, () => {
10   console.log(`Сервер запущен на порту ${port}`);
11 });
```

В этом примере в роуте, между буквой d и r можно указать любые символы в любом количестве.

Возврат HTML файла

До сих пор во всех примерах вы могли наблюдать, что HTML конечному клиенту передаётся в виде строки прямо в коде. Но HTML код может быть большим и держать его в строке будет не очень удобно. В express существует для этого возможность возврата файла в качестве ответа на запрос с помощью метода `res.sendFile()`. Допустим, существует скрипт сервера и в этой же директории находится файл `index.html`, который мы хотим использовать в качестве ответа.

```
1 .
2 |— node_modules
3 |— index.js
4 |— index.html
5 |— package.json
6 |— package-lock.json
```

Тогда код сервера будет выглядеть примерно так:

```
1 const express = require('express');
2 const path = require('path');
3
4 const app = express();
5
6 app.get('/', (req, res) => {
7   res.sendFile(path.join(__dirname, './index.html'));
8 });
9
10 app.listen(3000);
```

На седьмой строке происходит вызов метода `app.sendFile()` и в первый аргумент этого метода передаётся строка с путём к HTML файлу. Путь к файлу должен быть абсолютным, именно поэтому в примере используется модуль `path` и его метод `path.join()`. Также используется глобальное свойство `__dirname`, которое хранит информацию о том, в какой директории находится скрипт. Таким образом, передав в `path.join()` две составляющие: `__dirname` и название файла, метод возвращает полный путь к HTML файлу.

Статические файлы

Ещё express позволяет определить директорию, файлы которой будут доступны из браузера. Такие файлы называются статическими. Для того чтобы определить такую

директорию, необходимо использовать промежуточный обработчик `express.static()`. Давайте посмотрим на пример:

```
1 const express = require('express');
2
3 const app = express();
4
5 app.use(express.static('static'));
6
7 app.get('/', (req, res) => {
8   res.sendFile('static/index.html');
9 });
10
11 app.listen(3000);
```

На пятой строке используется метод `app.use()` для регистрации глобального промежуточного обработчика. В первый аргумент метода `app.use()` передаётся результат выполнения функции `express.static('static')` которая, в свою очередь, принимает в первом аргументе путь к директории, где хранятся статичные файлы. Обычно такую директорию называют `static`. Далее на восьмой строке в метод `res.sendFile()` передаётся путь к HTML файлу, текст которого необходимо вернуть в качестве ответа. При этом можно не указывать абсолютный путь к файлу, достаточно указать путь от директории, в которой хранятся статические файлы.

```
1 .
2 |— node_modules
3 |— static/
4 |   |— index.html
5 |— index.js
6 |— package.json
7 |— package-lock.json
```

Помимо всего прочего, статические файлы можно запросить прямо из браузера. Допустим, в директории `static` хранится два файла: `index.html` и `about.html`. Для того чтобы получить их содержимое, можно не писать отдельный роут, а просто указать имя файла прямо в URL таким образом: <http://localhost:3000/index.html> или <http://localhost:3000/about.html>.

Использование статичных файлов

Чаще всего, HTML страницы реализуются в паре с таблицами стилей. Для того чтобы в HTML использовать CSS файлы, можно воспользоваться механизмом раздачи статичных файлов. Допустим, у вас есть такой HTML файл:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-
    scale=1.0">
6   <title>Мой сайт</title>
7 </head>
8 <body>
9   <h1>Добро пожаловать на мой сайт!</h1>
10 </body>
11 </html>
```

Чтобы подключить стили к этому файлу, необходимо в заголовок добавить тег `<link/>`. В самом теге необходимо определить атрибут `href`, в котором нужно указать ссылку на CSS файл. Так как в директорию со статическими файлами можно поместить любые файлы, это как раз позволяет создать файл CSS в этой директории и ссылаться на него в HTML файле:

```
1 .
2 |— node_modules
3 |— static/
4 |   |— index.html
5 |   |— index.css
6 |— index.js
7 |— package.json
8 |— package-lock.json
```

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-
  scale=1.0">
6   <link rel="stylesheet" href="index.css">
7   <title>Мой сайт</title>
8 </head>
9 <body>
10  <h1>Добро пожаловать на мой сайт!</h1>
11 </body>
12 </html>
```

Также можно посмотреть содержимое файла CSS в браузере. Достаточно напрямую к нему обратиться <http://localhost:3000/index.css>.

Итоги

На этой лекции мы с вами разобрались с модулями и их видами. Поработали с некоторыми встроенными модулями. Узнали о некоторых глобальных свойствах и объектах. Поработали с express: разобрались с роутами, промежуточными обработчиками, возможностями работы с URL, а также поработали со статичными файлами. В следующей лекции мы с вами используем все полученные знания для того, чтобы реализовать небольшой проект на express.

Домашнее задание

- Инициализируйте NPM проект и попробуйте установить пакет express.
- Создайте файл index.js и импортируйте express.
- Попробуйте написать http сервер на express с одним обработчиком.

Дополнительные материалы

- Официальная документация о встроенных модулях Node.js <https://nodejs.org/dist/latest-v18.x/docs/api/>
- Официальная документация express <http://expressjs.com/ru/>

Словарь терминов

- **Модуль** – это файл, код которого можно использовать в других модулях
- **Функция `require()`** – это глобальная функция Node.js, которая используется для загрузки и кэширования модулей. Она принимает один аргумент – имя или путь к файлу модуля, который нужно загрузить, и возвращает экспортированные свойства из этого модуля.
- **Циклические зависимости** – это такие зависимости, при которых возникает ситуация, когда цепочка зависимостей начинается и заканчивается на одном и том же файле.